

操作服务插件

目录

一、 概述.....	4
二、 插件基类.....	6
插件接口及基类.....	6
创建插件.....	6
注册插件.....	7
三、 插件事件.....	9
onPreparePropertys 事件.....	10
onAddValidators 事件.....	11
beforeExecuteOperationTransaction 事件.....	11
beginOperationTransaction 事件.....	12
endOperationTransaction 事件.....	12
rollbackOperation 事件.....	13
setContext 事件.....	14
initializeOperationResult 事件.....	14
四、 案例演示.....	15
案例 1：自定义校验器，校验币别不能为空.....	15
案例 2：校验申请日期不能早于当前系统时间.....	17
案例 3：事务保护功能.....	18
五、 操作实践.....	21
需求 1：保存时验证申请数量.....	21
需求 2：审核时验证实际领用数量.....	21

需求 3: 编写操作插件, 当点击办公用品登记单保存按钮时, 校验物品明细 (单据体) 中采购时间不能晚于当前系统时间	21
需求 4: 编写操作插件, 当点击办公用品登记单保存按钮时, 自定义校验器, 校验币别不能为空	22
需求 5: 编写操作插件, 当点击办公用品登记单保存按钮时, 检查物品明细 (单据体) 计量单位是否为空	22

一、概述

系统预置了一批操作，可以绑定在界面菜单、按钮上；

用户点击按钮时，即自动执行这些操作，完成特定的操作功能。

这些预置的操作，可以根据是否写数据入库，分为两大类：

1. 表单操作：仅对表单界面及界面上的数据进行处理，不会写数据入库，如“新建”、“关闭”操作；
2. 实体操作：会写数据入库，只能配置在单据、基础资料上，如“保存”、“提交”、“审核”操作；

可以给单据上绑定的实体操作，单独开发操作插件，对操作写数据入库的过程，进行干预。

表单操作，不会写数据入库，只能针对表单界面及界面数据进行处理，不支持操作插件，但可以开发表单插件，对表单操作进行适度的干预；

与表单操作相关的表单插件事件，包括：

1. 菜单、按钮点击事件(beforeItemClick, itemClick, beforeClick, click)，可以取消操作的执行；
2. 操作执行前后事件(beforeDoOperation, afterDoOperation)，准备操作参数、接收操作执行结果刷新界面；

下表列出了系统预置的实体操作，只能对这些操作，开发操作插件（其他未列出的操作属于界面操作，不能开发操作插件）：

操作	功能说明
保存 (save)	把单据数据包中的数据，存入单据表格

保存并新增 (saveandnew)	把单据数据包中的数据，存入单据表格；保存成功后，清空界面数据，进入新增状态
状态转换 (statusconvert)	切换单据状态字段值，并更新单据表格
提交 (submit)	切换单据状态字段值为“ 审核中”，并更新单据表格
提交并新增 (submitandnew)	切换单据状态字段值为“ 审核中”， 并更新单据表格提交成功后，清空界面数据，进入新增状态
撤销 (unsubmit)	切换单据状态值为“ 暂存”， 并更新单据表格
审核 (audit)	切换单据状态值为“ 已审核”，并记录审核人，更新单据表格
反审核 (unaudit)	切换单据状态值为“ 暂存”， 并清除审核人，更新单据表格
禁用 (disable)	切换使用状态为“ 停用”， 并更新单据表格
启用 (enable)	切换使用状态为“ 启用”， 并更新单据表格
作废 (invalid)	切换作废状态为“ 作废”， 并更新单据表格
生效 (valid)	切换作废状态为“ 正常”， 并更新单据表格
删除 (delete)	从单据表格中，删除单据数据
空操作 (donothing)	特殊的实体操作，本身并不写单据入库；但会执行操作的全部过程，包括权限校验、写日志，支持操作插件开发，触发全部操作事

	件;
--	----

二、插件基类

插件接口及基类

单据操作插件接口为 `IOperationServicePlugIn`、`IOperationService`;

系统预置了一个操作插件基类 `AbstractOperationServicePlugIn`, 实现了上述操作插件接口;

自定义的操作插件, 扩展预置的操作插件基类 `AbstractOperationServicePlugIn` 即可。

```
package kd.bos.entity.plugin;
public class AbstractOperationServicePlugIn
implements IOperationServicePlugIn, IOperationService {.....}
```

插件基类 `AbstractOperationServicePlugIn` 内置了如下属性方法和本地变量, 供插件访问, 用以获取操作执行上下文

方法	说明
<code>billEntityType</code>	单据主实体
<code>operateMeta</code>	操作配置, 据此获知当前执行的操作信息
<code>operationResult</code>	操作结果, 可以向其中添加操作提示信息
<code>getOption()</code>	自定义操作参数字典, 可以包含各种自定义 操作参数

创建插件

自定义的单据操作插件, 需要扩展单据操作插件基类 `AbstractOperationServicePlugIn`。

```
public class OperationEventSample extends AbstractOperationServicePlugIn {.....}
```

操作编辑

操作类型

submit

操作编码

submit

操作名称

提交

参数设置

其他控制

操作后刷新字段

校验规则描述

合法性校验, 组合字段唯一性校验, 必录校验

服务端服务

权限项描述

提交

服务插件

[[{"_Type_": "Plugin", "MetadataId": "ZPYG29333", "ClassName": "kd.bos.plugin.sampl

操作前确认提示

操作成功后提示

取消

确定

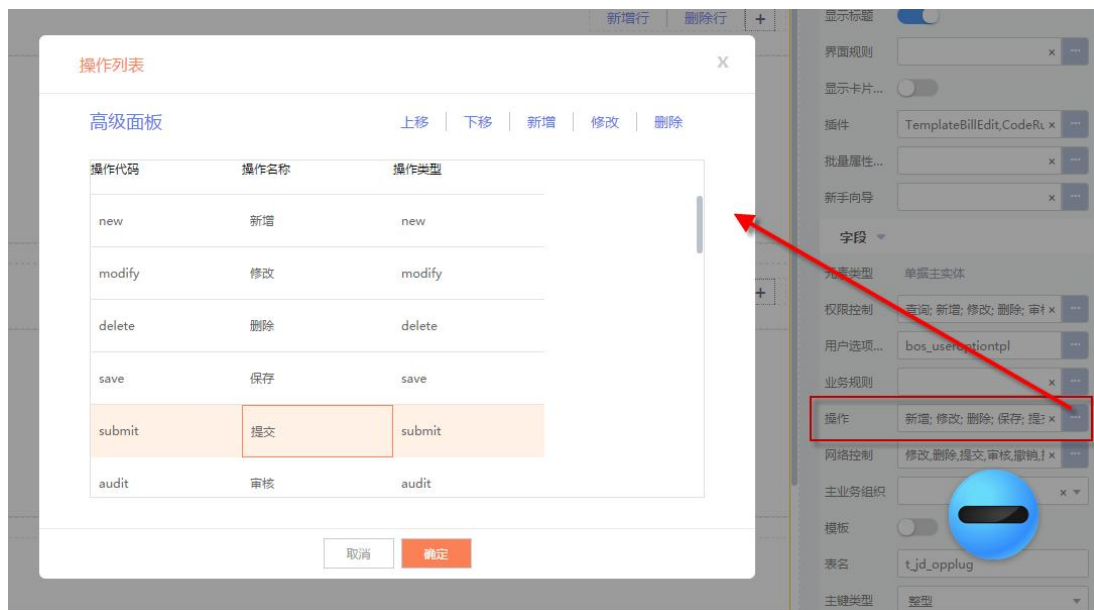
注册插件



已经开发好的单据操作插件，

通过如下步骤，绑定到操作上：

- 1.修改单据操作列表
- 2.修改操作的配置
- 3.绑定自定义操作插件



三、插件事件

单据操作插件，提供如下插件事件：

事件	触发时机
onPreparePropertyts	在单据列表上执行单据操作，系统需要先根据传入的单据内码，加载单据数据包；在加载单据数据包之前，触发此事件；插件需要在此事件，添加需要用到的字段；
onAddValidators	系统预置的操作校验器加载完毕，执行校验之前，触发此事件；
beforeExecuteOperationTransaction	操作校验通过之后，开启事务之前，触发此事件；
beginOperationTransaction	操作校验通过，开启了事务，准备把数据提

	交到数据库之前触发此事件；
endOperationTransaction	数据已经提交到数据库之后，事务未提交之前，触发此事件；
rollbackOperation	操作事务提交失败，事务回滚之后触发此事件；
afterExecuteOperationTransaction	操作执行完毕，事务提交之后，触发此事件；
setContext	设置上下文
initializeOperationResult	初始化操作结果集

onPreparePropertyys 事件

操作执行，加载单据数据包之前，触发此事件；

在单据列表上执行单据操作，传入的是单据内码；

操作引擎需要先根据传入的单据内码，加载单据数据包，其中只包含操作要用到的字段，然后再执行操作；在加载单据数据包之前，触发此事件；

在单据维护界面，执行单据操作时，传入的是单据数据包，不需要操作引擎自行加载单据，不会触发此事件；

插件需要在此事件，添加需要用到的字段；否则，操作引擎加载出的单据数据包，可能没有插件要用到的字段值，从而引发中断。

```
@Override
public void onPreparePropertyys(PreparePropertyysEventArgs e) {
    // TODO 在此添加业务逻辑
}
```

事件参数

```
public class PreparePropertyEventArgs  
    public List<String> getFieldKeys(): 需加载的字段标识
```

onAddValidators 事件

构建好操作校验器之后，执行操作校验之前，触发此事件；

插件可以在此事件，增加自定义操作校验器，或者去掉内置的校验器。

```
@Override  
public void onAddValidators(AddValidatorsEventArgs e) {  
    // TODO 在此添加业务逻辑  
}
```

事件参数

```
public class AddValidatorsEventArgs  
    public List<AbstractValidator> getValidators(): 预置的校验器  
    public void addValidator(AbstractValidator validator): 添加自定义校验器  
    public DynamicObject[] getDataEntities(): 本次操作、待校验的数据包
```

beforeExecuteOperationTransaction 事件

操作校验通过之后，开启事务存储数据之前，触发此事件；

插件可以在此事件，对已经通过校验的数据，进行整理，或者取消操作的执行。

这个事件触发时，还没有启动事务保护，请勿在此修改数据库数据。

```
@Override  
public void beforeExecuteOperationTransaction(BeforeOperationArgs e) {  
    // TODO 在此添加业务逻辑  
}
```

事件参数

```
public class BeforeOperationArgs extends OperationArgs
public List<ExtendedDataEntity> getSelectedRows(): 已经通过校验的单据
public DynamicObject[] getDataEntities(): 本次操作的全部单据，含校验失败的单据
cancel: 是否取消后续操作
public void setCancelMessage(String cancelMessage): 操作取消原因
```

beginOperationTransaction 事件

操作校验通过，开启了事务之后，还没有把数据提交到数据库之前触发此事件；

可以在此事件，进行数据同步处理。这个事件触发时，系统还没有调用 ORM 引

擎存储单据：此时访问数据库，拿到的单据数据是旧的；

单据数据包中的脏标志也没有被复位；

可能会执行失败的同步处理，如和第三方系统的对接，放在这个事件比较合适。

```
@Override
public void beginOperationTransaction(BeginOperationTransactionArgs e) {
    // TODO 在此添加业务逻辑
}
```

事件参数

```
public class BeginOperationTransactionArgs extends OperationArgs
public DynamicObject[] getDataEntities(): 已经通过校验的单据
public void setDataEntities(DynamicObject[] dataEntities): 设置允许继续操作的数据
public void setCancelFormService(boolean cancelFormService): 暂未使用
public void setCancelOperation(boolean cancelOperation): 是否取消操作
```

endOperationTransaction 事件

单据数据已经提交到数据库之后，事务未提交之前，触发此事件；

可以在此事件，进行数据同步处理；

这个事件触发，系统已经调用 ORM 引擎存储了单据，数据已经入库，单据数据包中的脏标志已经复位。

可能执行失败的同步处理，不适合放在这个事件执行，事务回滚比较麻烦。

特别说明：

系统会根据脏标志，判断出数据是否来自数据库，有没有更改：脏标志被复位后，调用 ORM 数据存储引擎，存储数据包，会被 ORM 引擎直接略过，不再更新入库。

```
@Override
public void endOperationTransaction(EndOperationTransactionArgs e) {
    // TODO 在此添加业务逻辑
}
```

事件参数

```
public class EndOperationTransactionArgs extends OperationArgs
{
    public DynamicObject[] getDataEntities(): 已经保存到数据库的单据
}
```

rollbackOperation 事件

操作事务提交失败，事务回滚之后触发此事件；

该方法在事务异常后执行，插件可以在此事件，对没有事务保护的数据更新进行补偿

```
@Override
public void rollbackOperation(RollbackOperationArgs e) {
    // TODO 在此添加业务逻辑
}
```

事件参数

```
public class RollbackOperationArgs
{
    public DynamicObject[] getDataEntitys(): 保存失败，被回滚的单据
}
```

afterExecuteOperationTransaction 事件

操作执行完毕，事务提交之后，触发此事件；

插件可以在此事件，对操作结果进行整理，或者执行其他无需事务保护的逻辑。

这个事件触发时，事务已经完成并提交，没有了事务保护，请勿在此事件更新数据库。

```
@Override
public void afterExecuteOperationTransaction(AfterOperationArgs e) {
    // TODO 在此添加业务逻辑
}
```

事件参数

```
public class AfterOperationArgs extends OperationArgs
    public DynamicObject[] getDataEntities (): 操作成功的单据
```

setContext 事件

设置上下文

```
@Override
public void setContext(MainEntityType billEntityType, Map<String, Object>
operateMeta, OperateOption option) {
    // TODO Auto-generated method stub
    super.setContext(billEntityType, operateMeta, option);
}
```

initializeOperationResult 事件

初始化操作结果集

```

@Override
public void initializeOperationResult(OperationResult result) {
    // TODO Auto-generated method stub
    super.initializeOperationResult(result);
}

```

四、案例演示

案例 1：自定义校验器，校验币别不能为空

```

public class OnAddValidatorsDemo extends AbstractOperationServicePlugIn {
    @Override
    public void onPreparePropertys(PreparePropertysEventArgs e) {
        super.onPreparePropertys(e);
        //将需要使用自定义校验器进行校验的字段加到列表中
        //e.getFieldKeys().add("qyrv_currency");
    }
    @Override
    public void onAddValidators(AddValidatorsEventArgs e) {
        e.addValidator(new MyValidator());
    }
    /**自定义校验器*/
    class MyValidator extends AbstractValidator {
        @Override
        public void validate() {
            // 获取扩展数据包
            ExtendedDataEntity[] datas = this.getDataEntities();
            for (ExtendedDataEntity dataEntity : datas) {
                DynamicObject object = (DynamicObject)
dataEntity.getValue("qyrv_currency");
                if (object == null) {
                    this.addErrorMessage(dataEntity, "币别不能为空");
                }
            }
        }
    }
}

```

注意：在单据列表上的"提交"按钮绑定此插件，可以触发 onPreparePropertys 事件。在单据编辑界面中的操作按钮上绑定此插件不会被触发 onPreparePropertys 事件。

需求：以出差申请单为例，当点击“保存”按钮时，校验币别不允许为空

注册插件查看演示效果

将插件注册在出差申请单的“保存”按钮上。新增单据数据，不选择币别，单击保存查看效果。

注册插件

插件路径*

kd.qyrv.tra.plugin.operationplugin.OnAddValidatorsDemo

描述说明*

保存时，验证币别是否为空

是否启用

☒

取消

确定

出差申请单

保存

提交

提交并新增

退出

基本信息

单据类型*

标准出差申请

单据编号*

TRIPREQ20230616-00008001

单据状态*

暂存

申请日期

2023-06-16

申请人

费用申请部门

环宇国际集团

费用承担公司

环宇国际集团

出差类型

驻外事务

币别

申请金额

1,000.00

事由

创建人

杨名

审核人

组织

研发部

币别不能为空

案例 2：校验申请日期不能早于当前系统时间

需求：以出差申请单为例，

校验申请日期不能早于当前系统时间

```
public class BeforeExcuteOperationTransactionDemo extends
AbstractOperationServicePlugIn {
    @Override
    public void beforeExecuteOperationTransaction(BeforeOperationArgs e) {
        super.beforeExecuteOperationTransaction(e);
        //获取当前数据包
        DynamicObject[] objects = e.getDataEntities();
        //获取申请日期
        Date reqDate = objects[0].getDate("qyrv_date");
        if(reqDate==null){
            e.setCancel(true);
            e.setCancelMessage("申请日期不能为空");
        }else{
            //获取当前日期
            Date currDate = new Date();
            if(currDate.compareTo(reqDate)>0){
                e.setCancel(true);
                e.setCancelMessage("申请日期不能早于当前系统日期");
            }
        }
    }
}
```

注册插件并查看效果

注册插件

插件路径*

kd.qyrv.tra.plugin.operationplugin.BeforeExecuteOperationTransactionDemo

描述说明*

校验申请日期不能早于当前系统时间

是否启用

取消

确定

出差申请单

1 申请日期不能小于当前日期

2 错误提示

保存

提交

提交并新增

退出

2 点击保存按钮

基本信息

单据类型*

标准出差申请

单据编号*

TRIPREQ-20230616-00008001

单据状态*

暂存

申请日期

2023-06-13

申请人

费用申请部门

环宇国际集团

费用承担公司

环宇国际集团

出差类型

驻外事务

币别

人民币

申请金额

¥ 1,000.00

事由

创建人

杨名

审核人

组织

研发部

1 当前系统日期为2023-06-16

案例 3：事务保护功能

在 beforeExecuteOperationTransaction 方法中编写如下代码没有事务保护功能

```

@Override
public void beforeExecuteOperationTransaction(BeforeOperationArgs e) {
    super.beforeExecuteOperationTransaction(e);
    for(DynamicObject dynamicObject:e.getDataEntities()){
        long pkValue = (long)dynamicObject.getPkValue();
        DB.execute(DBRoute.of("secd"),"update tk_qyrv_tripreqbill " +
            "set fk_qyrv_description='beforeExecuteOperationTransaction' " +
            "where fid='"+pkValue+"'");
    }
    throw new KDBizException("执行失败");
}

```

在 beginOperationTransaction 执行代码会有事务保护功能，我们编写插件来验证事务保护功能。

```

@Override
public void beginOperationTransaction(BeginOperationTransactionArgs e) {
    super.beginOperationTransaction(e);
    for(DynamicObject dynamicObject:e.getDataEntities()){
        long pkValue = (long)dynamicObject.getPkValue();
        DB.execute(DBRoute.of("secd"),"update tk_qyrv_tripreqbill " +
            "set fk_qyrv_description='beforeExecuteOperationTransaction' " +
            "where fid='"+pkValue+"'");
    }
    throw new KDBizException("执行失败");
}

```

将插件绑定到出差申请单保存按钮上,通过列表选中一条数据点击数据进入编辑界面后再点击保存

如果在 beforeExecuteOperationTransaction 方法中编写代码，虽然执行失败了，但是事务不会回滚。

出差申请单

新增

删除

保存

提交

提交并新增

打印

退出

执行失败

基本信息

单据类型*

标准出差申请

单据编号*

TRIPREQ-20230614-00007001

单据状态*

暂存

申请日期

2023-06-14

申请人

费用申请部门

环宇国际集团

费用承担公司

环宇国际集团

出差类型

临时出差

币别

人民币

申请金额

¥ 1,500.00

事由

客户现场开发

创建人

杨名

审核人

组织

cosmic-simple

五、操作实践

需求 1：保存时验证申请数量

编码实现办公用品领用申请单保存时，校验申请数量必须大于 0，且不能大于办公用品库存数量。

需求 2：审核时验证实际领用数量

- 1.审核时要检查实际领取数量不得大于对应物品的申请数量；
- 2.审核通过时，需要扣减对应物品的库存单库存数量。

需求 3：编写操作插件，当点击办公用品登记单保存按钮时，校验物品明细（单据体）中采购时间不能晚于当前系统时间

需求 4: 编写操作插件, 当点击办公用品登记单保存按钮时, 自定义校验器, 校验币别不能为空

步骤 1: 自定义校验器, 重写 validate()

```
class MyValidator extends AbstractValidator {  
    @Override  
    public void validate(){  
    }  
}
```

步骤 2: 重写操作插件中的 onAddValidators(), 添加自定义校验器

```
@Override  
public void onAddValidators(AddValidatorsEventArgs e) {  
    e.addValidator(new MyValidator());  
}
```

需求 5: 编写操作插件, 当点击办公用品登记单保存按钮时, 检查物品明细(单据体) 计量单位是否为空